

ActInf ModelStream 017.1 What's Wrong with Active Inference and How La

ModelStream | 1:05:47

Hello, welcome everyone.

It's March 21st, 2025, and we are here with a bunch of the developers of RxInfer and LazyDynamics.

We will be having a presentation followed by a discussion.

So, looking forward to everyone's comments here on the panel, and to anyone watching live, looking forward to your questions in the live chat.

Albert, to you for the presentation, and thanks everybody again for joining.

Hi, hi everyone. Yeah, thanks Daniel for this introduction.

Yeah, so for everyone, my name is Albert.

I'm a CEO of LazyDynamics, which is a spin-out from Eindhoven University of Technology.

And LazyDynamics essentially builds infrastructure for developing adaptive agents, such as active inference agents.

And today, I'm pleased to have almost full LazyDynamics team here with me.

So, yeah, let's get started.

I must say that actually, yeah, this talk is going to be admittedly provocative, perhaps even a clickbait.

But, yeah, let's just have some fun here.

And actually, I hope that many of active inference practitioners can relate to the pain point I'm about to discuss.

So, yeah, now I kind of expect that, yeah, some people won't like the way we pose the question.

And, yeah, I'm hoping, I'm actually not going to answer in the way that people might anticipate.

Because, yes, active inference is a theoretical framework, which is very elegant and very powerful.

But if you look at this from a practitioner perspective, it leaves much to be desired.

So, from a framework that claims to be superior to reinforcement learning, that promises to revolutionize AI, that's supposed to explain actually everything from bacterial behavior to human consciousness, what do we see in practice, right?

What do people who are getting excited actually see in practice?

Well, look at this.

Beautiful theory, groundbreaking equations.

And yet, this is sort of a typical reaction from a developer.

So, they're literally like running away.

And I don't think we can actually blame them for that.

Because, well, we've somehow managed to take one of the most promising frameworks for intelligent systems

and make it so complex that even a PhD student would think twice before implementing it.

So, and I just want to draw sort of a parallel and look at what's happening in the rest of AI.

Yeah?

So, five lines of code.

That's all it takes to access the state-of-the-art transformer models.

So, if you want to analyze satellite imagery, done.

Want to create a conversational agent, done.

Want to synthesize a video or audio, done.

So, half a page of the unique code.

But meanwhile, implementing an active inference agent.

Well, let me just walk you through this.

So, what it takes.

First, you read through hundreds of pages of dense mathematical theory.

Then you spend months on implementing some custom algorithms.

Then you debug some mysterious convergence issues.

And after all that work.

What do we have?

Well, more often than not, it's going to be a T-Maze example.

Or like maybe a mountain car.

Yeah?

I mean, come on.

What is this?

You just made me read dozens of pages of heavy math.

You can solve me with moving pixels on the screen.

Right?

So, this is quite a leap between I am better than reinforcement learning.

And I can do a T-Maze better than reinforcement learning.

So, I mean, come on.

So, we are like in 2025 now.

So, the rest of AI is solving like really interesting world problems.

And like having really interesting demos.

And we are still celebrating when our agents can turn left or right.

So, and truth to be told.

I mean, I think my personal experience is that, yeah.

Like each time I'm getting into this active inference literature.

It's, to put it mildly, not always very much reproducible.

Or like half of it is not.

There is not much to reproduce.

So, the point here is that this needs to be changed.

So, and not just for convenience.

But because this complexity is, I think, actively holding back the entire field.

We are basically limiting the active inference to a tiny group of theoretical specialists.

When it should be in the hands of every AI developer.

And the saddest part, actually, of all this.

That people who used to be passionate about the theory.

They just give up on it.

And we just see it all around.

Like in the community.

So, how can we fix that?

How can we actually make this as accessible as transformers?

So, for starters, we must recognize what makes active inference agent unique.

Yeah?

So, well, according to the theory, it's obviously not the inference mechanics.

Right?

Because the inference mechanics is same across all agents.

It's just free energy minimization.

So, what's left for us, like these three lines of code, is basically the model and constraints.

So, this is the only unique part.

This is the only, like, these unique three lines of code, if you wish, which defines the agent.

So, it should be only about model definition and constraints.

And that's exactly what we are trying to do with Ericsson-Fur.

So, here we develop the framework that we are continuously developing.

This framework that allows you to provide a model, constraints, and run the inference guided by the free energy minimization.

And I'll talk about this in the next slide a bit more.

And what we actually can see is that once you have the framework right, or let's say almost right, because it's a development project, you can streamline the development of these agents by orders of magnitude.

And, seriously, like, when I started my PhD, I spent a few years trying to build an audio counterpart of a teammate's demo,

so that people can now build in just a few hours.

So, that's kind of the order of magnitude you should think of.

And, yeah, just to zoom in a little bit on some of the examples.

Now, basically, if you want to build a drone that does planning through inference,

it would just take about 10 lines of code to define the drone model and then, like, run inference to get this nice path planning.

So, here you see, like, two 3D drone examples.

One is flying in the rainy day, another one is in the sunny day, and another one is, like, 2D drone, which I'm more excited about because it actually fills this reactivity aspect of our framework.

So, you could see that we are changing the environment and the drone doesn't hold.

It continues to sort of reinfer its policies and its states in order to reach this goal no matter the environmental changes.

So, yeah, 10 lines of code.

So, if you want to, yeah, same applies to locomotion control.

So, there are two examples, one for robotic arm, another one for this, the name for this robot.

I forgot about basically this swinging monkey robot, right?

So, let's phrase it this way.

So, again, define your model, run inference.

If you want to have multi-agent navigation in a dynamic environment with collision avoidance and speed constraints,

that would take you, like, 15 lines of code.

Not thousands, just 15.

It's quite a boost, I would say.

So, same, yeah, another just a few demos from our repository.

Again, if you want to integrate deep learning into the, into a RxNferi, it's also possible.

You can learn the dynamics and, like, predict the future sequences all in one go.

So, it's not two separate function calls.

You just define it.

You just call the inference within the same model.

So, eight, yeah, here's, like, about eight lines of code.

And, yeah, surely, yeah, how could we avoid mentioning a better version of T-MASIS?

So, that's something not publicly available, though.

But that's something we've been working on.

There's kind of an agent that needs to solve a mini-grid.

So, there is an agent that needs first to locate the key, then take the key, then figure out that he needs to open the door and then go to the exit.

So, that's something that works.

This is the only example which is not publicly available yet, but it will be soon.

So, and, yeah, before I go into how we are doing that, I just want to say also that these examples themselves, they're hardly as impressive as reinforcement learning demos or, to critique, NPC demos.

But they're better than, I would say, the stigmatized examples of active inference supremacy.

So, yeah, I think that it's still ongoing, but it's going pretty fast.

And, yeah, so how we do it?

What's our approach?

So, we do it through essentially four key innovations.

So, the first one is most important is like free energy minimization.

It's automated.

No manual derivations needed for a very large class of models.

And they're like, they're composable.

But that's basically why we adhere to factor graphs,

because a factor graphs is a very convenient underlying model representation,

which allows us to essentially seamlessly combine different models into bigger models

or the other way around, decompose bigger models into smaller submodels.

And another thing, yeah, of course, we adhere to asynchronous data

such that we can handle and facilitate this real-time adaptation to changing environments.

And the really key thing here is reactive message passing technology.

And in our opinion, this is the actual key to successful implementation of functional agents

or like functional active inference agents.

And the argument here is that, see, the world, the real world,

I mean, it doesn't start and stop at the agent's convenience.

So, the agent must react, adapt, act upon environmental data streams.

So, if something unexpected happens,

and the world always brings some unexpected things to you,

and, yeah, you essentially, you have to adapt.

So, we must compute our policies.

We must compute our posteriors, no matter the failure.

So, that was what you could see with that little drone demo.

And so, this is what our implementation of...

I think, so, this is something that, like,

what the other implementation of active inference aren't getting quite right.

I think, and actually, yeah, to be fair,

most of the reinforcement learning libraries adhere also

to these imperative updates of their policies and so on.

Nevertheless, I don't want to give the impression

that the current implementation of reactive message passing works

as we wish it would.

It's super fast.

We are, like, beating, in terms of inference, many other frameworks.

But we also recognize how it can be improved.

And the truth to be told, it can be improved.

We can actually make it faster and better.

That is something we are working on as well.

So, what's the state of RxInfer today?

RxInfer was and will remain the open source library

that has more than now 10 years of cumulative development time.
We are about 10 active developers from LaceDynamics and BiasLab.
BiasLab is the laboratory where we spun out from.
And, of course, ActiveInference Institute,
who is helping us with the development of the graphical interface,
with demos.
Like, there's really lots of help coming from ActiveInference
that we appreciate.
So, we have over 730 stars on GitHub
across reactive-based packages.
And more than 300 monthly downloads now.
I mean, these are not the craziest numbers you could think of.
But I think here's why these numbers actually matter.
They show that despite all the barriers,
despite all the complexity,
despite having to learn Julia,
despite having to learn probabilistic programming,
people are still pushing to use ActiveInference.
They push to use RxInfer in general.
And we are seeing this...
And just try to imagine,
like, if we are seeing that much interest,
when it is hard,
I just want you to imagine
what would happen when we make it easy.
And that's kind of exactly what we are building towards.
So, before RxInfer,
in the early days of my PhD,
and I think even prior to that,
I had to write quite some code,
and actually do quite some math.
Now, with Bioslab and ReactiveBase community,
we push it to the one page of model definition,
and everything else is automated.
But at LazyDynamics,
we also recognize that even that is too hard.
So, even that is really not going to streamline
the development of ActiveInferenceAsian,

because there is a huge Python community,
and, like, people want to integrate
our models with their models and so forth.
So, that's where LazyDynamics came in.
So, we are actually going to hide the complexity
even further and wrap the models
into a nice open IP available for Python, Rust, and so on.
So, we do recognize that model specification
is still hard for many users.
So, in the beginning,
we will provide models for users and developers.
We will start simple,
and then we will make it more and more interesting
and complex with the vision
that people can actually be able to submit their models,
host their models,
and we will just provide the API to trigger those.
So, a bit of a teaser.
It's not available for mass use.
It's just something we are testing internally.
So, how that's going to look like?
Well, so, one would basically have to load the model
from our pool of RxInfer models,
and then run inference slash planning,
take actions, and learn.
So, we will do it now in the separate API calls for clarity,
but, I mean, the mechanics in the early hood are the same.
So, it's just free energy minimization.
We just realized that, yeah,
people get a hang of it faster if we just pose it like this.
But, yeah, it's a work in progress.
We plan to release this API in early Q2.
So, it won't be as flexible as RxInfer,
but it will expose endpoints,
and we will make it better over time.
Yeah, and this is the team that has this ambition,
like this is LazyDynamics team.
But I must say that there are more people behind this now.

So, there are more people behind RxInfer development.

This is just solely the LazyDynamics team.

Yeah, but there are more developers,
as I said, outside this LazyDynamics team.

So, yeah, just, yeah, I think last few bytes for me is that,
yeah, I think that ActiveInference needs to move finally
from like this beautiful, beyond beautiful theory
and into practical applications.

We are showing that it's possible,
and the framework is indeed revolutionary,
and I think that now it's time to make it usable.

So, yeah, thank you for listening,
and here I'd like to stop
and actually invite people for questions.

Awesome. Thank you.

If any of the others here just would like to go around,
say hello if they'd like any first thoughts,
and meanwhile, anyone in the live chat can write questions.

Which order do you want to go, Daniel?

I mean, I can start.

I think it's fantastic what Albert
and the team is developing.

I still believe ActiveInference is the future,
also to reinforcement learning problems,
and it's the tool set, user-friendly tools that's lacking.

So this is an initiative, I think,
that hopefully will get us closer to, yeah,
to recognition of ActiveInference
as the underlying framework for future intelligent agents.

And, yeah, everybody is invited to contribute,
to critique, to give feedback.

So, yeah, we're very open in terms of
how we should develop towards the future,
because we don't have all the answers ourselves.

But, yeah, this is our push, first push,
to make it as simple as working with Transformers.

Thank you.

Dimitri, do you want to add a few words

as a core architect of Rixenferr?

Well, I can only add that you were a bit wrong

saying that it's not publicly available.

Well, actually, the API is almost publicly available.

We have released the initial version of it.

We just didn't advertise it as much,

and it doesn't really have all the models that we have.

But we are planning to release the API

that Albert has mentioned for free for now,

so everyone can experiment

and everyone can break it as fast as possible,

because as soon as we fail,

we know how to fix it.

So fail fast approach,

we will deliver new models,

we will test it, we will fail,

we will improve.

And for this,

we essentially need more people to try it out,

because we have our internal use cases,

we tried out,

it works on our internal cases,

but there are many other people

who also want to try this approach.

So we will co-develop the model,

so we will release them for free for now

for the free API,

and we'll see how it goes.

And this is just the tip of the iceberg,

right?

So I think the team of Lazio Dynamics,

we have already had the pleasure to work with the API.

So a year ago,

we all didn't think it would be possible,

but especially with a lot of efforts from Dimitri,

we're now able to call RxCifer from Python.

So this should integrate very easily

with existing reinforcement learning toolboxes out there.

And I've already had the opportunity to play with it.

I'm playing with it also before this live stream,

and I'm really enjoying it so far.

So basically merging the infrastructure

that's available in Julia

with everything that's already available in Python.

And I think that's just the tip of the iceberg.

There's a lot more in store coming up real quick.

Yeah.

So everyone would want to try it out.

Also just contact us,

reach out,

share contact details,

whatever we want.

Because I think we should all adopt this approach.

Maybe just Bart,

one follow-up question there.

Like in integrating through Python calls to RxCifer,

where do you see that plugging in

for somebody who wants to use it

amidst the other machine learning methods

they're already deploying?

Like what would that look like?

How would they conceptualize of their pipeline

and where these kinds of probabilistic models would fit in?

I think these probabilistic models themselves

are very important when you are trying to model a process

that involves uncertainty

making these autonomous systems.

So in those regards,

I think that's where current machine learning framers often lack

is that usually there's this black box neural network.

I think personally that having a probabilistic take on specific aspects,

being just in the latent space of a neural network, right?

I think that could benefit a lot of enterprises

and a lot of products,

making them more reliable and robust.

And because most of these products have like

some backend,
some machine learning models already in Python,
I think that would be amazing
to integrate RxCifer from Julia
into their existing infrastructure.
Yeah, just on top of that,
there is an opportunity that we,
well, not the opportunity,
there are a few demos people are trying out
is basically to connect their LLM-based agents
to connect them to the backend of RxCifer.
That's also something feasible.
And apart from it,
so there are many problems indeed
that can be tackled
with probabilistic programming languages, right?
Like you can,
there are many at the table,
like IMC and NumPyro.
And yeah, people use them as components
into their bigger like machine learning pipelines.
And here the,
well, now you basically can,
can do it with RxCifer,
just basically without leaving
your Python environment,
you can call RxCifer
to provide you with some uncertainty estimates
and yeah, confidence over the answers
for other of parts
of your machine learning pipelines.
You can also add maybe here
a little bit to expand
how we can integrate with LLMs
because we can imagine that LLM
is a sort of a,
is a source of noisy data.
For example,

you can ask questions to LLM
to give some numerical answers,
but you should not really trust it.

So you may,
you may think of it
as the noisy observation
of a real data.

So,
and you can create a model
that takes into account
and basically treat this
as your observations.
And then the underlying
probabilistic model
will infer the real parameters
of the process.

So I think,
yeah,
RxCifer can be integrated
very naturally
in the existing pipeline.
So it shouldn't really be,
or it's not necessary
to have just RxCifer
in your pipeline.

And it can be integrated
with like other data collection libraries
or the existing machine learning libraries.

So,
yeah,
I think,
but
it,
it excels
when you really need
to create a plan
given a noisy set of observation
and given the model

of the process.

Okay.

Cool.

Ismail,

if you'd like to add anything

and then

otherwise,

maybe to Frasier

and then we'll go to some

pre-written questions

in the live chat.

Yeah,

sure.

I'd like a couple

of things to say.

First of all,

the new

RxCifer server,

I think,

will add

extraordinary capabilities

for both Python developers

and for Julia developers

that are

struggling

find the

platform

to develop

agents

that are not

hallucinating.

Yeah,

so for that,

big,

many thanks to Dimitri

for

realizing this.

And,

yeah,
I would like to invite
everyone
who is
keen on learning
about machine learning
and developing
agents
or developing
models
for
whatever task
they have in mind.
to try out
RxCifer
and
engage with us
their problems
and
their experiences,
whether it's
negative
or negative.
That will be a very valuable
formation for us.
And that's
mostly it.
something.
If I can add,
I see a little bit
like a big focus
on Python,
but Albert didn't really
focus on that much.
But
the way we
created RxCifer
server

is with the
OpenAPI specification
that allows
to use the same
API from
any language
that supports
OpenAPI,
which includes
basically all languages
that exist,
including Julia.

So it is possible
to use Julia
for RxCifer
server
or Python
or JavaScript
or Rust
or even C.

And if I'm not mistaken,
even microcontrollers,
they do have
some sort of
a layer
between this
OpenAPI
specification.

So it really opens doors
not just for Python
developers,
but
also from the browser
we can now use RxCifer
because JavaScript
is also supported.
So it really opens doors
like

for experimentation
in many fields.

Awesome.

Brazier,
you prepared
some excellent
comments.

So
however you'd like
to continue,
go for it.

Cool.

Thank you.

I should just
clarify,
I'm not a developer
on RxCifer.

I'm an open source
contributor
and I'm a PhD student
at Glasgow,
but I'm not part
of the core team.

So I have,

I think, Daniel,
what I'll do

is I'll condense
those questions

I had down into two,
hopefully just to save
a bit of time.

The first question
is about active inference
more generally
because this talk was,
well, thank you.

First of all,
thank you for the talk.

It was very enlightening.

It was excellent stuff.

Thank you, Albert.

But about active inference,

the problems

with active inference

and how RxCifer

is kind of addressed

to solve them.

It seems to my mind

basically that we have issues

with real-time inference.

We have issues

with robust inference

and scalable inference

and then even beyond

that scaling,

say,

like the generative model

itself to be able

to take care

of more sophisticated

inference queries.

Those kind of three things

seem to me

to be the current issues

with active inference

and making it real-time,

robust,

and reliable.

I mean,

RxCifer has effectively

been kind of the best

tool in the game

for addressing those concerns

because of its very nature.

But can anyone speak

to, let's say,

perhaps how continuing
developments of RxCifer
are going to assist
in those three kind of domains?
And I have a follow-up question
about RxCifer server.

So I know it's very broad,
but alas,

I had to ask it.

Dimitri,
do you want to answer?

Yeah, I can answer.

It's a great question.

So right after RxCifer server,

what we plan,

we want to change

and refactor the core

of RxCifer

to make the points

that you mentioned

even easier to do,

even easier to scale

and faster

to run inference,

but also

to make it easier

to debug

and to contribute

for other people

because also

with the experience

with you,

we noticed

that it's quite difficult

to contribute

because the,

first of all,

it's written in Julia,

so not all of people
know Julia,
but also because
it has started
as a PhD project
and certain parts
weren't really
designed very well.

So we will take
a step back
and we'll,
given all the knowledge
that we have now
and given all the feedback
and given all the goals
that we and other people
want to achieve,
we'll redesign
the entire core
of RxFMP
and this potentially
will open the doors
also to run
the inference
on GPUs,
for example.

So that's
how transformers
basically scaled
themselves, right?

They started
to use GPUs at all.
at large scale.

So we have
big plans
to improve
the scalability
even further

and make the inference
even faster
and easier
to maintain
and debug
and contribute
for other people.

Excellent.

That sounds like
a small,
you know,
problem.

Did I cut someone else
off there?

I sensed another voice.

Yeah,
well,
that was me.

I mean,
you didn't really
cut me off.

But yeah,
perhaps I can add
to that a bit.

I'm more of a big picture
person.

And so maybe also
for people
that are just wondering
about active inference
and are still
in other areas
of machine learning
like reinforcement learning
or deep learning,
why would they be
interested?

So I think

most people agree
that the Bayesian
inference
is a correct framework
for dealing
with uncertainty.

But the big thing
is the computational
cost
and that limits
us in scaling
this all up.

Right?

So people
have mentioned
ideas like,
well,
maybe we should
do planning
as inference.

This is possible
with ARX and FERR.

And we're working
on stuff

that we will
probably reveal
also in the next
half year

where I think
it will be a nice
surprise

in how to do
planning as inference.

And then there
is an even
more broader
idea

that this idea

that you shouldn't
do Bayesian inference
because of the
computational cost
which limits
the scaling.

I think it's
almost the opposite
around.

By doing
Bayesian inference,
you have access
to where the
uncertainties are.
and therefore
you also
have access
to when you
just should
stop computing
because you
shouldn't compute
more accurately
than is needed.

When your data
is,
when you have
a large variance
about your data
or your variables,
you don't need
very accurate
computing.

So I think
once we're done
with Bayesian inference,
I mean,
not just lazy

dynamics,
but the whole
community,
it's going to
turn out
that it is
not,
that it's going
to be because
of doing it,
doing Bayesian
inference,
that we can
do actually
scale up
intelligent agents
to very large
problems.

So Bayesian inference
will not be
the hindrance,
it will be a
catalyst for
scaling up.

So that's
the,
I think,
why people
should get
interested
in active
inference,
even if they're
still in
just researching
at this
moment.
How is it?

Excellent.

Thank you,

Bart.

I'm very,

very,

very keen

on the

entire

methodology

here.

Just my

last question

and then

I'll open

it up

to everybody

else.

So about

RxInfer

server,

I myself,

I played

around creating

a few active

inference agents,

bona fide

agents with

RxInfer,

which is

currently

difficult

because RxInfer,

you know,

it specifies

the inference

procedure and

automates

the way

that that

happens.

The actual

design of an

agent is

kind of

another thing.

That's,

you know,

the inference

procedure is

one part,

but at the

moment,

it has,

I and

anybody else

making an

agent has

to do

everything

kind of

ad hoc.

We have

to specify,

well,

if an

action happens

here,

that affects

the environment

in some

specific way.

It's all

very ad hoc

and so on.

So my

understanding

is that
the agent
API,
as part
of RxInfer
server,
is going
to perhaps
ameliorate
those issues
or standardize
the way
in which
we talk
about how
an action
influences
the environment
and vice
versa.

Is that,
let's say,
a big focus
of the API
or is that
just now
a part
of the API
in the sense
that we can
now do
much broader
things like
have endpoints
in Python
and so on.
So it's kind
of a larger

concern than
just this API.

So any
comments?

Yeah,
sure.

I think
Dmitry would
definitely add
upon that.

Just from
my side,
there is
actually
ongoing
discussion
that I
forgot
to mention
in our
GitHub
repository
about
the
standardizing
API.

I think
one of
the things
which we
are also
striving
with our
RxInfer
server
and generally
with
streamlining

this
development
of
agents
is
actually
to
standardize
how
the API
should
look
like.
That
there
should
be
a
certain
function
that
should
be
triggered
in
a
reactive
way
or
in
the
loop
if
you
prefer
imperative
style
coding.

But
there
should
be
really
sort
of
we
want
to
impose
one
way
or
few
ways
to
actually
doing
that.
So
yeah,
another
thing
is
just
want
to
welcome
everyone
to
this
discussion
on
RxInfer
repository
where
you,

Fraser
and
other
people
from
Active
Inference
Institute
also
participated.
But
the
goal
of
course
is
the
end
goal
is
to
bring
in
the
key
components
which
are
needed
to
run
agents
in
the
way
we
run
transformers

I mean

low

code.

Okay,

I see.

Thank you.

Well,

Daniel,

I'll hand it back

to you.

Maybe we can

look at some

questions in

the live

chat.

I'd just

like to

say thank

you very

much.

This

is an

incredible

project.

I'm

very

enthusiastic

about

the

future.

Thank

you.

Awesome.

I'll read

some

questions

in the

live

chat
and
then
we'll
see
if
we
have
time
for
some
of
the
more
pre-discussed
questions
or any
other
questions
popping
up.
Okay,
so two
questions
on
computational
resources.
Indian
Goat
wrote,
can you
scale
to
large
data
sets?
And
Jeff

Beck
wrote,
I
would
have
thought
that
memory
costs
would
have
been
the
!
bottleneck
for
scaling
care
to
comment?
And
so to
that
I'll
just
sort
of
also
add
a little
bit
of
my
own
computational
resourcing.
Just
how do

you find
that
storage,
memory,
and CPU
cost
scale?

Is it
possible
to do
static
analysis?

How do
you find
that
resourcing
scales
in
comparison
with
alternative
methods?

Yeah,
I can
start
and
maybe
you
can
enroll
a bit
on
this
VQV8
for
example
how
can

we
scale
further
but
when
we
talk
about
scaling
we
can
look
at
it
from
two
perspectives
the
first
is
the
amount
of
variables
we
have
in
the
model
right
so
we
have
a
million
of
variables
but

all
of
them
would
say
univariate
variables
so
for
this
kind
of
scaling
Ericsson
already
does
a
pretty
good
job
because
we
use
factor
graphs
it's
only
limited
by
memory
in
your
computer
and
I
personally
I
run

inference
for
2 million
univariate
variables
and
it
has
been
done
under
like
5
minutes
or so
it
was
pretty
easy
but
another
aspect
of
scalability
is
indeed
the
dimension
of
each
individual
variable
so
for
example
all
the
multivariate

and
this
is
indeed
this
is
still
a
hard
problem
so
if
you
have
a
very
large
state
vector
then
indeed
you
need
to
propagate
their
covariances
if
they
are
Gaussian
and
this
might
be
tough
for
which

now
we
prefer
to
use
VQVA
basically
we
compress
the
state
into
a
smaller
dimensional
state
Albert
can
talk
a
little
bit
more
about
that
or
for
example
we
use
techniques
similar
to
mean
field
that
when
you

approximate
the
covariance
matrix
with
diagonal
matrix
which
also
reduces
the
memory
requirements
there
are
many
techniques
that
we
can
use
the
thing
is
that
RxInfer
doesn't
really
choose
it
for
you
it's
more
like
a
playground
for

a
user
for
now
where
they
can
choose
different
approximation
methods
or
different
techniques
but
RxInfer
doesn't
restrict
you
here
you
can
even
try
your
own
approximation
techniques
but
that
would
require
you
to
write
your
own
message

passing
rules
which
we
again
we
noticed
that
many
users
they
cannot
do
that
because
it's
hard
it's
hard
and
it
requires
a lot
of
knowledge
on
how
to
do
that
yeah
just
to
add
on
that
about
this

what
was
the
question
from
Jeff
was
like
about
this
memory
cost
yeah
so
I'm
going
to
answer
that
in
three
parts
so
the
first
one
is
to
use
sorry
to
say
that
non
Bayesian
techniques
right
so

or
not
entirely
Bayesian
so
and
that's
something
like
actually
active
inference
community
should
have
been
doing
like
earlier
I
think
so
basically
what
you
could
do
to
scale
is
what
Mitri
said
you
can
use
semi
Bayesian

approaches
like
variation
auto
encoders
to
compress
to
basically
cast
your
high
dimensional
vector
representation
into
a
low
dimensional
vector
right
so
that's
the
basic
thing
you
can
actually
think
of
you
can
do
it
in
also
multiple

levels
it
doesn't
have
to
be
only
in
the
likelihood
you
can
do
it
in
the
state
so
I
mean
as
you
move
in
your
in
the
hierarchy
of
your
model
you
can
embed
this
variational
encoders
one

after
another
so
that's
one
thing
and
another
thing
is
the
downside
here
is
that
you
require
pre-training
right
so
there
is
some
pre-training
involved
not
cool
sometimes
sometimes
it's
okay
another
way
of
tackling
that
is
to

actually
use
hierarchical
model
representation
so
you
can
encode
your
state
see
in
as
a
sort
of
hierarchical
multi-scale
model
right
so
you
don't
have
to
when
you
have
a
grid
world
for
instance
you
don't
have
to

yeah
basically
model
all
these
components
of your
grid
into
one
state
right
so
like
it
will
end
up
yeah
so
it
doesn't
scale
well
so
we
know
a
few
frameworks
that
adhere
to
that
and
well
I
think

it
will
not
progress
much
but
if
you
actually
manage
to
have
sort
of
multi-scale
representation
of
this
grid
world
right
so
you
can
have
sort
of
like
a
location
in
terms
of
you
can
break
it
down

right
so
first
you
can
have
a
big
grid
world
which
you
can
divide
by
four
elements
and
then
you
can
zoom
in
zoom
in
zoom
in
so
basically
you
would
have
sort
of
this
granular
representation
of your

state
so
and
this
scales
pretty
well
and
another
thing
is
like
well
it's
kind
of
what
Bert
touched
upon
right
is
this
vision
going
forward
is
indeed
like
to
use
this
reactive
approach
to
solve
this
scaling

once
and
forever
right
so
it's
basically
the
idea
here
is
not
to
spend
much
computational
resources
just
basically
spend
as
much
as
you
can
so
but
yeah
that's
something
work in
progress
yeah
and
I
know
if
Bert

you
want
to
add
something
on
top
of
it
about
this
computational
cost
I
think
you
have
something
well
yeah
I mean
we just
discussed
that
right
I mean
I think
ultimately
a
Bayesian
framework
will help
computational
costs
because
you have
access
to

how
much
precision
you
want
for
your
computations
and
you
know
that
means
you
can
stop
early
you
can
the
nice
thing
about
a
message
passing
process
is
reactive
message
passing
is
that
you
can
interrupt
it
at

any
time
and
take
the
result
as
it
is
and
if
you
wait
longer
you
get
more
accurate
result
but
if
you
stop
early
then
you
have
a
less
accurate
result
but
it
may
be
okay
and
you

know
in
Bayesian
framework
you
have
access
to
how
accurately
you
want
to
compute
at
the
higher
levels
in
these
in
these
hierarchical
models
you
don't
need
a
lot
of
accuracy
right
if
I
want
to
park
my

car
I'm
not
going
to
worry
about
the
centimeters
of
where
I
actually
park
I
just
want
to
find
a
parking
spot
right
so
that's
I'm
talking
about
tens
twenties
of meters
right
so
it's
so
it's
partly
in the

architecture
and
it's
partly
of
doing
smart
inference
in
the
end
yeah
and
two
more
things
here
I
just
realized
that
well
you
it's
so
I've
mentioned
that
yeah
you
can
use
non-Bayesian
approaches
to
train
your
save

a
rational
but
actually
now
there
are
ways
of
training
your
neural
networks
in
online
manner
also
so
it's
not
inherently
prohibitive
to
actually
do
learning
and
inference
in
the
goal
there
yeah
there
are
some
developments
in

that
part
of
that
world
as
well
and
I
think
the
last
piece
here
is
that
hardware
for
Bayesian
inference
is
also
emerging
so
it's
something
that
people
are
also
working
on
it's
not
at
mass
scale
yet

but
there
will
be
hardware
breakthroughs
I
think
that
would
be
specifically
targeting
this
applications
for
of
Bayesian
inference
if
there
are
no
other
comments
or
questions
just
now
I
would
like
to
follow
up
on
this
issue

of
hierarchical
modeling
maybe
we've
exhausted
it
just
now
but
it
appears
to me
as
it
does
to
many
people
that
this
issue
of
hierarchy
is
going
to
be
very
important
for
just
active
inference
going
forward
this
is

very
much
how
biological
organisms

are
arranged

they
have
characteristic
units

that deal
with
their
own
time
scales
and
they
are
connected
to
some
other
level
above
and
level
below
I
know
that
in
ARX-INFER
you can

create
bonafide
submodels
within
other
model blocks
and so
on
but
is
there
I'm
interested
in
the
sort
of
landscape
of
hierarchical
modeling
for
ARX-INFER
in
particular
are
there
things
that
you
guys
would
like
to
highlight
or
emphasize
in

that
regard
for
users
like
me
I
think
Dmitry
maybe
you
can
just
tease
a
bit
about
this
idea
of
multi
scale
or
hierarchical
models
are
available
right
now
but
what's
not
available
out
of
the
box
is

basically
as
you
said
different
updating
schemes
for
different
levels
so
that's
something
Dmitry
can
elaborate
that's
what
I
mentioned
about
rewriting
the
core
of
our
our
our

our
our
to
also
support
this
feature

more
naturally
so
I
can
imagine
indeed
that
we
have
a
really
huge
model
with
big
hierarchies
but
we
don't
really
want
to
update
deep
hierarchy
as
often
as
we
update
for
example
the
layer
that
is
connected

directly
to
our
observations
because
yeah
as
Bert
mentioned
for
certain
tasks
!
we

do
less
accurate
predictions
but
in the
current
implementation
it's
a bit
hard
to
do
it's
like
it
requires
a lot
of
hacks
so
we
tried

it
once
in
principle
it's
possible
to
do
but
it's
like
user
unfriendly
so
that's
also
one
of
the
directions
that
want
to
improve
when
you're
writing
a
XNinfer
to
better
support
hierarchical
models
to
make it
easier
to

initialize
hierarchical
models
to
make
it
easier
to
support
different
update
rates
for
different
hierarchical
awesome
I'll
ask
a few
more
questions
in
the
chat
so
Michael
Lennon
asks
the
difficulty
of
implementation
is a
great
provocation
point
what
have

been
use
cases
where
you
have
seen
some
of
the
greatest
benefits
from
your
approach
and
what
have
been
your
most
surprising
findings
great
question
I
think
if
what
are
the
greatest
I
look
so
I
think
technically

if
you
just
go
to
what
was
most
surprising
I
think
it's
just
the
fact
that
you
use
the
same
technique
same
inference
approach
same
back
same
back
end
to
tackle
all
sorts
of
different
problems
like
here

there's
many
problems
on
control
of
course
but
like
it
goes
way
beyond
that
right
so
there
are
like
some
more
classical
examples
on
latent
autoregressive
models
gaussian
processes
sensor
fusion
and
missing
data
so
I
think
the

benefit
of
our
approach
in
particular
like
first
of
is
that
it
works
close
to
real
time
and
certain
applications
is
definitely
real
time
not
everywhere
but
yeah
I
think
just
the
like
the
fact
that
it's
so

versatile
in
terms
of
different
domains
while
kind of
connecting
all of
them
under
the
same
umbrella
principle
right
so
that's
I think
the most
powerful
finding
of
RxNinfer
in
particular
yeah
so
it's
also
a reason
why
just
generally
this approach
is not
super

popular
I think
in engineering
communities
right
of
inference
hasn't
is not
a big
name
in
the
engineering
communities
because
as
engineers
and we
are
an
engineering
group
we
are
used
to
start
with
a
problem
and
then
you
go
to
the
literature

you
find
10
possible
solutions
and
you
pick
one
and
you
change
it
a bit
and
now
you
have
11
solutions
approaches
this
is
entirely
the
other
way
around
for
every
whatever
the
problem
is
we
have
the
same

approach
we
build
a
model
we
do
inference
it's
one
solution
approach
to
any
model
any
problem
right
so
it's
completely
the
opposite
way
of
thinking
of
problem
solving
for
an
engineer
and
it's
very
hard
I
know

this
is
with
the
students
in
my
class
it's
very
hard
to
make
that
switch
from
one
problem
has
lots
of
possible
solutions
to
we
have
one
solution
method
for
all
the
problems
that
is
that's
very
hard

and
but
it
works
right
I
think
this
is
the
right
way
but
it's
going
to
be
difficult
to
make
that
switch
in
engineering
communities
awesome
just
again
to
footnote
and
include
some
of
these
great
questions
and

welcome
people
into
this
continuing
conversation
and
open
source
development
so
a few
more
okay
urban
avsec
wrote
does
rx
infer
and
if
so
to
what
extent
support
structure
and
parameter
learning
to
train
and
improve
models
automatically
okay

so
yeah
that's
a good
question
I wouldn't
say that
there is
automatic
way of
doing
that
you can
there are
I think
examples
on
structure
learning
to some
extent
and
look
so
one
thing
is
that
people
so
sometimes
people
mean
different
things
by
referring
to

structural
learning
if
they're
talking
about
finding
this
kind
of
this
discrete
program
search
to
automatically
discover
what is
the right
graph
for your
particular
problem
problem
it's
I think
there's
a lot
of
ongoing
research
it's
not
set
in
stone
how
to

do
it
and
I
think
we
have
good
intuition
we have
good
understanding
on how
to
prune
the
models
based
on
this
well
essentially
obviously
Bayesian
model
evidence
approaches
so you
can
definitely
use
that
within
Rx
Infer
but
how
do

you
extend
the
model
how
you
grow
sort
of
your
graph
I
think
this
is
still
an
open
question
and
an
interesting
one
and
as
for
another
way
of
looking
at
actually
structured
learning
right
or
you
could

think
of
non-parametric
methods
including
Gaussian
processes
for instance
and
those
are also
going
to be
integrated
in
Rx
Infer
I
think
yeah
hopefully
this
year
but
the
idea
is
that
yeah
you
can
just
basically
discover
this
distribution
over
your

over
the
functions
that
drive
that
explain
the
dynamics
of
the
environment
so
we
are
certainly
working
on that
from
different
angles
and
there
is
also
BiasLab
that
BERT is
leading
that
also
tackles
some
of
these
issues
as
for

parameter

learning

yeah

I

think

this

is

sort

of

I

wouldn't

say

solved

right

nothing

is

like

100%

solved

but

I

think

that's

something

we

are

good

at

there

is

no

problem

with

parameter

learning

and

as

I

said
pruning
that's
available
yeah
awesome
yeah
and
batch
and
streaming
inference
for
parameter
learning
okay
Andrew
Pachea
wrote
thank

you
for
the
updates
how
are
you
currently
handling
multi
agent
agent
environment
loops
iteratively
e.g.
creating

an
agents
at
the
top
of
the
script
and
using
loops
or
functions
for
creating
and
storing
them
as
opposed
to
hard
coding
each
individual
agents
so
what
kind
of
design
patterns
have
you
been
working
with
or

do
you
think
are
relevant
for
multi
agent
situations
yeah
multi
agent
yeah
thanks
thanks
for
the
question
multi
agent
is
of course
like
the
pinnacle
sort
of
all
these
inference
problems
I
think
there
well
you
one
way

is
like
you
need
to
hack
it
through
your
loop
another
way
is
actually
like
what
you
see
here
in
this
multi
agent
path
planning
is
that
it's
a
sort
of
super
agent
so
there
is
kind
of

a
meta
agent
that
encapsulates
the
models
of
all
other
agents
and
then
they
share
this
information
now
the
problem
is
that
it
doesn't
scale
well
I
would
say
because
well
like
four
agents
five
agents
ten
agents

fine
but
then
like
you
get
right
you
get
it
so
I
think
one
of
the
ways
I
mean
Dmitry
probably
can
can
also
add
upon
that
but
what
you
could
do
like
as
for
the
design
pattern

is
you
run
your
agent
for
instance
like
speaking
of
this
path
planning
so
you
run
your
agent
into
the
you
let
it
go
into
your
environment
and
you
only
interact
when
you
are
in
the
proximity
of

other
agents
so
there
is
not
need
to
share
any
information
if
you
are
far
away
from
each
other
and
like
once
you
are
getting
closer
you
can
start
sharing
the
information
you
can
sort
of
fuse
the

models
figure
out
the
conflict
and
then
just
like
depart
so
that's
that's
one
of
the
things
I
can
suggest
but
Andrew's
question
there
is
no
we
we
didn't
think
of
how
we
didn't
let
me
put
it

this
way
we
didn't
standardize
the
API
call
for
multi
agents
so
not
yet
all
right
great
and
one
more
question
from
live
chat
Jeff
back
wrote
my
question
is
can
the
parameters
of
the
flows
and
the

VAEs
be
learned
in
Rx
and
fur
or
are
they
imported
if
it
is
learned
is
that
by
traditional
gradient
descent
well
I
think
this
is

if
Ismael
is
you
you
can
yeah
maybe
I
can
talk

a bit
on
that
so
it's
possible
to
update
weights
or
biases
of
the
neural
network
that
is
parametri
in
your
VAE
or
your
flow
within
Rx
and
fur
however
if
you
want
to
use
deep
and
wide
networks

then
you
need
the
GPU
architecture
that
Dimitri
was
referring
to
so
for
small
size
neural
networks
very
easy
you
can
use
Rx
and fur
to
tune
the
weights
but
for
larger
scales
in
its
current
weights
trained
outside

of the
model
and
then
use
these
parameters
for
prediction
purposes
but
it is
certainly
possible
to
incorporate
the
training
of
neural
networks
within
the
model
macros
awesome
okay
in
sort of
our
closing
round
here
one
of
the
pre-written
questions

was
what
are
the
upcoming
milestones
of
lazy
dynamics
and
what
is
your
vision
so
just
on
that
though
or
however
else
you
want
to
give
some
last
thoughts
if
each
of
the
developers
would
just
want
to

give
any
last
thoughts
what
are
they
looking
forward
to
this
year
where
might
we
be
by
the
mid
point
of
the
year
by
whatever

milestone
or
timeline
you
want
to
point
to
and
that
will
be

a
great
closing
route
so
how
about
Albert
and
then
continuing
on
with
the
developers
to

to

Thank you.

Thank you.

Thank you.

Sorry.

That's it was about this approach in particular to enterprise use.

So we see a lot of opportunities.

I'm sorry.

I made it just an audio mistake.

So people might've lost audio for a few seconds.

Just, could you just start again?

Start again?

Yes.

Sorry.

From where?

Just, just, yeah.

Yeah.

Sorry about that.

Yeah.

So two things then, like the very first thing is that I definitely want to invite, so LacedDynamics is going to support the open source development of RxInfer.

So that's our priority on the development side.

We are actually going to, we are planning to run a sort of, not a hackathon, but the development event, co-development event probably this summer.

And yeah, so I just want to invite more developers to help and testers to help with agent development, to help with demos.

even though you are not into RxInfer, sorry, active inference yet, right?

So everyone is welcome.

And one thing for us, I think for this year from the development perspective is of course to bring, to put RxInfer to the state where it becomes easier to contribute, easier to contribute for, to the sort of, to the engine, not just on the kind of this demo side, but also to improve the engine, to make it faster, more scalable.

So that's something we are, yeah, looking for.

And I want everyone to invite to that venture.

And then I think from LacedDynamics perspective specifically, yeah, we're inviting the partners and like forward thinking people, individuals, right?

Who just want to connect and want to join us in this adventure of building this, well, very scalable Bayesian inference solutions.

I think there's a lot of opportunities and possibilities for education and the enterprise.

And yeah, you name it.

So yeah, that's, that's some closing words from me.

Okay.

So from, from the research side, I think a big issue for active inference is the computation of expected free energy

and computation of policies, but minimizing expected free energy.

And I think they're making some nice progress there.

And we, I think we will also release that this year somewhere at the conference and in the package on how to do this with, with our sin for it in efficient way, of course.

From the development side is, of course, as has been mentioned already, we are releasing a RxNF server for now for everyone to use to play.

And as soon as it's finished, it's never finished, but of course, improve it.

But as soon as it has some stable form, we will rewrite the core of RxNF.

And then we also have other pending topics like integrating Gaussian processes very easily, making it very easy to use mixture models,

making it easy to introduce hierarchies in the model.

So yes, we have a lot of plans for the development.

We also have plans to improve the visualizations, right?

Since, since, since Fraser helped a lot with that, and we have, we have plans to improve that even further.

I think all the improvements and milestones were already mentioned.

So I'm not going to add more to that.

I think we already have quite a big list.

Nonetheless, there are a lot of surprises coming up.

So definitely stay tuned.

I think we already gave a good impression of what direction we'll be working towards.

But along the way, there will always be more cool stuff coming.

So definitely follow us everywhere.

And I think that's already, yeah, it gives a good impression of what we'll be up to.

Awesome.

Ismila?

Yeah, it's hard to add on top of Albert, Bart, all the team said, but as a personal, interest, I'm very interested in stochastic differential equations and continuous time systems.

And how we numerically approximate these systems, identify their parameters and so on.

And this is one of the research that we are doing also at RxFIR and how we can integrate them.

So that's a big research plan for myself.

And yeah, it's too much on the plate.

And we could enjoy contribution from everyone to this big plate of ideas.

So Frasier, where does that take you?

I just want to say I'm extraordinarily excited about continuing to contribute with you guys and develop the package further.

Maybe even I could help us in forays into the refactor of the package.

I know that can be intimidating, but we'll see.

So, yeah, I look forward to working with you guys in the future.

I look forward to it all.

Thank you.

Thanks, Faisal.

Awesome.

Thank you again for joining and just another chapter in the journey and in learning and in the application of Active Inference.

So, until next time.

Thank you all.

Thanks, Daniel.

Thanks, Saul.

Bye-bye.

Thank you.

Bye.

Bye, Ryan.

Bye.